

Universidad Simón Bolívar
Departamento de Computación y Tecnología de la Información
CI4321 – Computación Gráfica I
Sep–Dic 2026

Profesor: Jorge Palacios
Equipo: Sudo Render
Estudiantes: Baudilio Velasquez – 18-10665
Luis Isea – 19-10175

Tarea 01

Informe de implementación y análisis

Para esta tarea partimos del triángulo del laboratorio y fuimos agregando las otras figuras por separado. Primero construimos el cuadrado, después el hexágono y finalmente el círculo. Al terminar, conectamos las cuatro figuras con un botón que permite recorrerlas y dejamos el cambio de color en otro botón independiente. Así podemos cambiar de figura sin perder el color que habíamos seleccionado.

Todo lo dibujamos con WebGL2 y la biblioteca `FixedFunction` que se utiliza en el curso, sin modificar su implementación. A continuación explicamos las decisiones que tomamos y cómo se relacionan con las preguntas del enunciado.

1. ¿Cuántos vértices necesitaría el cuadrado con `gl.drawArrays(gl.TRIANGLES, ...)`? ¿Por qué el *index buffer* ahorra datos?

Aunque un cuadrado tiene cuatro esquinas, WebGL lo termina dibujando como dos triángulos. Si usáramos `drawArrays` con `TRIANGLES`, tendríamos que colocar tres vértices para el primer triángulo y otros tres para el segundo. Eso nos daría seis entradas en el arreglo de posiciones, a pesar de que dos de esas posiciones serían repetidas. Es decir, no aparecen seis puntos diferentes en la figura; simplemente tenemos que volver a mencionar algunos para completar los dos triángulos.

En nuestro caso preferimos guardar las cuatro esquinas una sola vez, dentro del arreglo `SQUARE`, y definir aparte el orden en que se usan. Para eso creamos `SQUARE_INDICES` con los valores `[0, 1, 2, 0, 2, 3]`. Los primeros tres índices forman un triángulo y los otros tres forman el segundo. Ambos comparten la diagonal que va del vértice 0 al 2 y mantienen el orden antihorario que solicita la tarea. Después los dibujamos usando `drawElements`.

El ahorro está en que el arreglo de posiciones no tiene que repetir las esquinas que comparten los triángulos. Los índices también ocupan espacio, pero suelen ser más pequeños que volver a almacenar todas las coordenadas. En este cuadrado la diferencia no es enorme; aun así, nos ayuda a entender por qué usar índices se vuelve especialmente útil cuando una figura tiene muchos triángulos conectados entre sí.

2. Para el hexágono, ¿cuántos vértices e índices usa cada método: `TRIANGLE_FAN` vs. `TRIANGLES`? ¿Cuál elegimos y por qué?

Para formar el hexágono dividimos su superficie en seis triángulos que parten del centro y llegan a dos puntos consecutivos del borde. Si dibujáramos cada uno por separado con `TRIANGLES` y `drawArrays`, tendríamos que enviar tres vértices por triángulo, es decir, 18 entradas y ningún índice. El centro aparecería repetido seis veces, y varios puntos del contorno también se repetirían. Otra posibilidad sería usar `TRIANGLES` con `drawElements`: así podríamos almacenar solamente siete vértices diferentes (el centro y los seis del borde), pero necesitaríamos 18 índices para describir los seis triángulos.

Nosotros elegimos `TRIANGLE_FAN` porque se adapta bastante bien a la forma que queríamos construir. Con este método enviamos primero el centro, luego recorremos los seis puntos del borde y repetimos el primer punto para cerrar la figura. En nuestra implementación eso da ocho entradas en el arreglo de vértices y cero índices. WebGL toma el centro como punto común y va formando un triángulo con cada pareja de puntos consecutivos del contorno.

Nos pareció una solución más sencilla de mantener, sobre todo porque no necesitamos escribir a mano los vértices de cada triángulo. La función `createFanVertices` calcula los puntos usando el ángulo correspondiente a cada división, y después usamos directamente ese arreglo para dibujar. Además, esa misma idea nos sirvió para hacer el círculo en el siguiente paso.

3. Con `SEGMENTS = 6`, ¿cuántos triángulos tiene el círculo? ¿Cuántos vértices e índices enviamos?

Con `SEGMENTS = 6`, nuestro círculo tiene seis triángulos. Esto ocurre porque cada segmento representa una parte del borde y se conecta con el punto central para formar un triángulo. Como usamos `TRIANGLE_FAN`, no tenemos que definir los seis triángulos de manera individual ni crear un buffer de índices: basta con enviar los puntos en el orden correcto.

La función `createFanVertices` comienza agregando el centro, después calcula seis puntos alrededor de la circunferencia y, al final, vuelve a agregar el primero de esos puntos para cerrar el contorno. Por lo tanto, enviamos ocho entradas de vértices y ningún índice. Aunque hay solamente siete posiciones distintas, una de ellas se repite al final del arreglo para completar el último triángulo.

Para la entrega dejamos `SEGMENTS = 64`, que es el máximo permitido. La construcción sigue siendo la misma, pero el contorno tiene muchos más lados pequeños y por eso se parece mucho más a un círculo. Con ese valor nuestra función genera 64 triángulos y 66 entradas de vértices, sin índices.

4. Al poner `SEGMENTS = 6`, ¿qué ocurre al comparar el círculo con el hexágono y por qué?

Para comprobarlo, cambiamos temporalmente el valor de `SEGMENTS` de 64 a 6 y volvimos a mostrar el círculo en la aplicación. El resultado nos llamó la atención porque, aunque el botón seguía indicando "Figura: círculo", en pantalla aparecía un hexágono. Después lo comparamos con la figura del hexágono y vimos que sus contornos coincidían. En la Figura 1 dejamos las capturas del círculo con 64 segmentos y del resultado al reducirlo a seis.

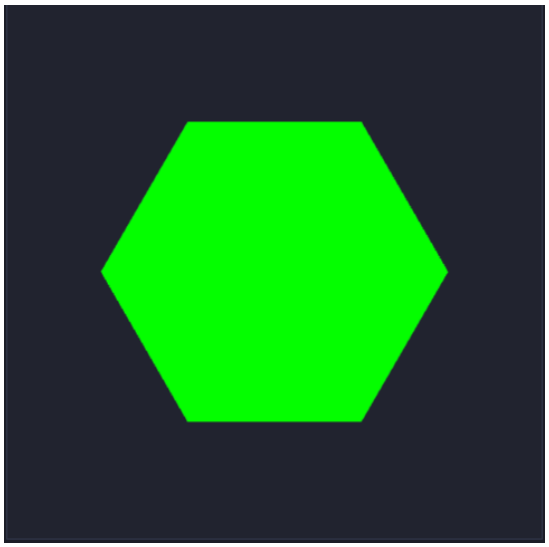
Esto tiene sentido por la manera en que construimos las figuras. Tanto el hexágono como el círculo utilizan nuestra función `createFanVertices` y el mismo radio. Al dejar seis segmentos,

colocamos solamente seis puntos sobre el borde; cada uno queda separado del siguiente por un ángulo de 60 grados. Al unirlos con líneas rectas, obtenemos exactamente los seis lados del hexágono, aunque internamente sigamos dibujando seis triángulos con `TRIANGLE_FAN`.

Con esta prueba entendimos mejor que el círculo no aparece como una curva perfecta desde el principio: lo aproximamos usando triángulos. Cuando aumentamos la cantidad de segmentos, los lados se hacen más pequeños y el contorno se ve cada vez más redondo. Al finalizar la comparación, devolvimos `SEGMENTS` a 64, que es el valor usado en la entrega.



Círculo con 64 segmentos.



Círculo con 6 segmentos.

Figura 1. Comparación que realizamos al cambiar la cantidad de segmentos.

5. Si las figuras no tienen color en sus buffers, ¿de dónde sale el color? ¿Qué pasaría si el VAO habilitara `ATTRIB.COLOR`?

Los arreglos de nuestras figuras contienen únicamente coordenadas de posición. No agregamos valores RGB para cada vértice porque en esta tarea todas las partes de una figura deben verse del mismo color. En cambio, cuando presionamos el botón de colores, escogemos rojo, verde o azul y enviamos ese valor a `ff.setColor` antes de dibujar. Así el color se aplica a toda la figura, sin importar si estamos mostrando el triángulo, el cuadrado, el hexágono o el círculo.

Esto funciona porque cada VAO tiene habilitado solamente `ATTRIB.POSITION`. Mientras `ATTRIB.COLOR` esté deshabilitado, WebGL puede usar el valor constante que establece `ff.setColor` para ese atributo. Por eso tampoco necesitamos modificar la geometría cada vez que cambiamos de color: las posiciones siguen siendo las mismas y lo único que actualizamos es el estado del color antes de dibujar.

Si habilitáramos `ATTRIB.COLOR`, WebGL intentaría leer valores de color desde un arreglo de atributos asociado al VAO, en lugar de usar aquel valor constante. Como nosotros no preparamos un buffer de colores ni configuramos cómo leerlo, la figura dejaría de tener una fuente válida de color por vértice y podríamos encontrarnos con errores al dibujar. Por esa razón dejamos el atributo de color deshabilitado, tal como pide el enunciado.

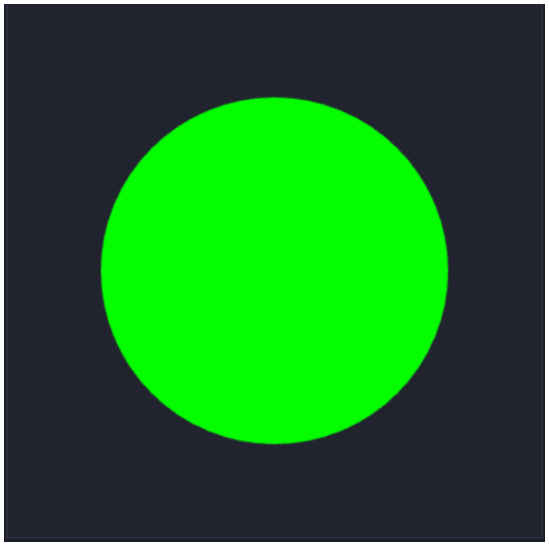
6. ¿Qué le pasa al círculo al usar un `CANVAS_SIZE` más ancho y por qué?

Para esta comprobación cambiamos temporalmente el ancho interno del canvas de 480 a 720 píxeles, mientras que la altura se mantuvo en 480. Además, ajustamos en el CSS la proporción visual a `3 / 2`, porque normalmente la página tiene `aspect-ratio: 1` y fuerza al elemento a verse cuadrado. Con ambos ajustes pudimos observar claramente qué sucede al dibujar la misma figura en un área más ancha.

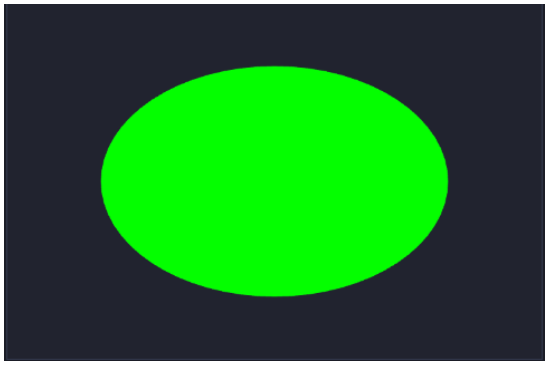
Como se ve en la Figura 2, el círculo dejó de verse redondo y tomó una forma parecida a una elipse, estirada hacia los lados. Nos pareció interesante que no modificamos los vértices del círculo ni el radio con que los calculamos. Lo único diferente era la relación entre el ancho y la altura del espacio donde WebGL lo mostraba.

Esto ocurre porque trabajamos directamente en *clip space*, sin aplicar matrices que compensen la proporción del canvas. Al transformar las coordenadas a píxeles, la misma distancia horizontal ocupa más espacio que una distancia vertical equivalente, de modo que la figura aparece ensanchada. La prueba también nos mostró por qué era importante revisar el CSS: si manteníamos la proporción cuadrada en la presentación, el navegador podía disimular visualmente el cambio del área interna.

Después de comparar ambas imágenes, restauramos el canvas de 480 por 480 píxeles y dejamos nuevamente `aspect-ratio: 1`. Así el círculo conserva su forma redonda en la versión que entregamos.



Canvas de 480 × 480.



Canvas de 720 × 480.

Figura 2. El círculo antes y durante la prueba con el canvas más ancho.